In [1]:
```
import pandas as pd
import numpy as np
```

计算熵

$Entropy = -\sum_{i=1}^{n} p_i \log_2 p_i$

In [2]:
```
# 计算熵
def entropy(data, target_col):
    counts = data[target_col].value_counts()
    p = counts / len(data)
    entropy = -np.sum(p * np.log2(p))
    return entropy
```

计算信息增益

$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$

使用 numpy.unique() 函数计算每一个 feature 下的分类和数量

subset 利用 pandas 进行切片，提取出每一个 value(分类) 的数据，然后计算熵

In [3]:
```
# 计算某一个 feature 的信息增益
def gain(data, feature, target_col):
    total_entropy = entropy(data, target_col)
    values, counts = np.unique(data[feature], return_counts=True)

    weighted_entropy = 0
    for i in range(len(values)):
        subset = data[data[feature] == values[i]]
        weight = counts[i] / np.sum(counts)
        weighted_entropy += weight * entropy(subset, target_col)

    gain = total_entropy - weighted_entropy
    return gain
```

决策树的构建

递归算法构建决策树

如果所有样本属于同一类别，则返回该类别作为叶节点

如果没有特征可用，则返回样本中出现次数最多的类别作为叶节点

预测结果

结果保存到 tree 字典当中，如果遇到新特征，默认返回训练集中出现次数最多的类别作为预测结果

In [4]:
```
def build_tree(data, features, target_col):
    if len(np.unique(data[target_col])) == 1:
        return np.unique(data[target_col])[0]

    # 返回最多数的类别
    if len(features) == 0:
        return data[target_col].mode()[0]

    gains = [gain(data, feature, target_col) for feature in features]
    best_feature_idx = np.argmax(gains)
    best_feature = features[best_feature_idx]

    tree = {best_feature: {"__default__": data[target_col].mode()[0]}}

    remain_features = [f for f in features if f != best_feature]

    for value in np.unique(data[best_feature]):
        subset = data[data[best_feature] == value] # 把 best_feature 的每个
        if len(subset) == 0:
            tree[best_feature][value] = data[target_col].mode()[0] #取出最
        else:
            tree[best_feature][value] = build_tree(
                subset, remain_features, target_col
            )
    return tree


def predict(tree, instance):
    if not isinstance(tree, dict):
        return tree

    feature = next(iter(tree))
    feature_value = instance[feature]

    if feature_value in tree[feature]:
        return predict(tree[feature][feature_value], instance)

    return tree[feature].get("__default__", np.nan)
```

In [5]:
```
if __name__ == "__main__":
    train_data = pd.read_csv("Data/train.csv")
    predict_data = pd.read_csv("Data/predict.csv")

    target = "weather"
    features = train_data.columns.tolist()
    features.remove(target)

    decision_tree = build_tree(train_data, features, target)

    predictions = []
    for _, row in predict_data.iterrows():
        pred_label = predict(decision_tree, row)
        predictions.append(pred_label)

    predict_data["weather"] = predictions
    predict_data.to_csv("result.csv", index=False)
    print("预测结果：", predictions)
    print("结果储存到 result.csv")
```

```
预测结果:  ['cloudy', 'sunny', 'sunny', 'cloudy']
结果储存到 result.csv
```